

Day 5 Practice Assignments: Functions, Scopes & RegEx

Objective

Design modular functions with varied parameters, enforce boundary check parameters using keyword-only scopes, implement stateful closures using `nonlocal` variables, and extract structured text properties using compiled RegEx capture groups.

Easy Assignments

Assignment 1: CDAC Cafeteria Discount Calculator

Scenario

The CDAC Cafeteria needs a modular pricing function to calculate student bills. The cafeteria offers main combo meals, optional side-dishes, standard tax rates, promotional discounts, and delivery charges.

Problem Description

Write a function named `calculate_cafeteria_bill(base_price, *items, tax_rate=0.05, discount=0.0, delivery_fee=0.0)` that calculates the final bill.

1. `base_price` (float): The cost of the main combo meal.
2. `*items` (floats): A variable-length positional argument list representing prices of additional side items.
3. `tax_rate` (float): The tax percentage (default `0.05` for 5% tax). This **must** be a keyword-only parameter.
4. `discount` (float): A percentage value (e.g., `10.0` represents a 10% discount, default `0.0`) applied directly to the subtotal before taxes.
5. `delivery_fee` (float): A flat shipping surcharge added to the final total after taxes (default `0.0`).

Calculation Rules:

1. Sum the `base_price` and all side item prices (`*items`) to compute the raw subtotal.
2. Deduct the discount from the raw subtotal to compute the discounted subtotal:

$$\text{Discounted Subtotal} = \text{Raw Subtotal} \times \left(1 - \frac{\text{discount}}{100}\right)$$

3. Compute the tax value by multiplying the discounted subtotal by `tax_rate`.
4. Add the tax and `delivery_fee` to the discounted subtotal to get the final bill.
5. Return the final total rounded to **2 decimal places**.

Example Walkthrough

```
# 1. Standard meal, no sides, default tax, no discount, no delivery
total1 = calculate_cafeteria_bill(100.0)
```

```
# Subtotal = 100.0
# Tax = 100.0 * 0.05 = 5.0
# Return: 105.00

# 2. Meal with sides, custom tax rate, 10% discount, flat delivery fee
total2 = calculate_cafeteria_bill(100.0, 20.0, 30.0, tax_rate=0.08,
discount=10.0, delivery_fee=15.0)
# Raw Subtotal = 100.0 + (20.0 + 30.0) = 150.0
# Discounted Subtotal = 150.0 * (1 - 10/100) = 135.0
# Tax = 135.0 * 0.08 = 10.8
# Final Total = 135.0 + 10.8 + 15.0 = 160.8
# Return: 160.80
```

Assignment 2: Academic Email Validator

Scenario

The CDAC academic portal needs to validate user registration submissions so that only valid academic emails ending in `.edu` or `.res.in` are registered.

Problem Description

Write a function `validate_academic_email(email)` that checks if a string is a valid academic email address using a regular expression.

- The email must satisfy the following syntax rules:
 - Username:** Must consist only of lowercase letters, numbers, dots, and underscores (`a-z`, `0-9`, `.`, `_`). It must contain at least one character.
 - Separator:** Must contain exactly one `@` symbol.
 - Domain:** Must consist of lowercase letters, numbers, dots, and hyphens (`a-z`, `0-9`, `.`, `-`).
 - Suffix:** The domain must end with either `.edu` or `.res.in` (and nothing else).
- The regular expression must perform an exact match of the entire string (use boundary markers `^` and `$`).
- The function must return `True` if the email matches all criteria, and `False` otherwise.

Example Walkthrough

```
print(validate_academic_email("arham.khan@cdac.res.in")) # Output: True
print(validate_academic_email("lisa_stud12@mit.edu"))    # Output: True
print(validate_academic_email("vinod@gmail.com"))        # Output: False
(invalid suffix)
print(validate_academic_email("ALICE@college.edu"))      # Output: False
(contains uppercase letters)
print(validate_academic_email("bob@mit.edu.com"))        # Output: False
(does not end in .edu or .res.in)
```

Medium Assignments

Assignment 3: Corporate Directory Search & Scraper

Scenario

You are writing a parser to extract formatted employee phone records from unstructured text files. Employee phone numbers are formatted in multiple ways across the directory.

Problem Description

Write a function `scrape_directory_phones(directory_text)` that extracts phone records from text and returns a structured list of dictionaries.

1. The function must detect phone numbers matching any of the following three formats:
 - `AAA-PPP-LLLL` (e.g., `123-456-7890`)
 - `(AAA) PPP-LLLL` (e.g., `(123) 456-7890`)
 - `AAAPPPLLLL` (10 consecutive digits, e.g., `1234567890`) where `AAA` represents the area code (3 digits), `PPP` represents the prefix (3 digits), and `LLLL` represents the line number (4 digits).
2. Design a single compiled RegEx pattern to parse all three formats using **capture groups**.
3. For each match found in `directory_text`, build a dictionary with the following keys:
 - `"area_code"`: String containing the extracted 3 area code digits.
 - `"prefix"`: String containing the extracted 3 prefix digits.
 - `"line_number"`: String containing the extracted 4 line number digits.
 - `"formatted"`: A normalized phone string in the format `"(AAA) PPP-LLLL"`.
4. Return a list of these dictionaries. If no phone numbers are found, return an empty list.

Sample Input

```
directory = "Contact HR at 123-456-7890 or the helpdesk at (987) 654-3210.  
Direct line is 5558881234."
```

Expected Output

```
[  
  {"area_code": "123", "prefix": "456", "line_number": "7890",  
   "formatted": "(123) 456-7890"},  
  {"area_code": "987", "prefix": "654", "line_number": "3210",  
   "formatted": "(987) 654-3210"},  
  {"area_code": "555", "prefix": "888", "line_number": "1234",  
   "formatted": "(555) 888-1234"}  
]
```

Scenario

An AI classification pipeline processes raw data inputs. Each raw input is a tuple of string annotations describing a product name, its price, and rating. The pipeline needs to clean, filter, and sort these records.

Problem Description

Write a function `process_dataset(dataset)` that processes a dataset using built-in higher-order functions (`map`, `filter`) and `lambda` expressions:

- `dataset` is a list of tuples containing string records. Example:

```
[("Laptop", "Price: 1200", "Rating: 4.8"), ("Phone", "Price: 800", "Rating: 4.5")]
```

- Your pipeline must execute the following sequential steps:
 - Parsing:** From the incoming raw tuples, extract the product name (string), numeric price (float), and rating (float). (You can use string splitting or RegEx to isolate the numeric values).
 - Filtering:** Use `filter()` with a `lambda` function to keep only items with a parsed price less than or equal to `1000.0`.
 - Mapping:** Use `map()` with a `lambda` function to transform the filtered entries into dictionaries of the following structure: `{"product": <name>, "price": <float_price>, "score": <float_rating>}`.
 - Sorting:** Sort the resulting list of dictionaries in **descending order of their score** using `sorted()` with a `lambda` key selector. If two items have the same score, their relative order does not matter.
- The function should return the sorted list of dictionaries.

Sample Input

```
data_input = [  
    ("Laptop", "Price: 1200", "Rating: 4.8"),  
    ("Phone", "Price: 800", "Rating: 4.5"),  
    ("Mouse", "Price: 25", "Rating: 4.7"),  
    ("Charger", "Price: 15", "Rating: 4.2")  
]
```

Expected Output

```
[  
    {"product": "Mouse", "price": 25.0, "score": 4.7},  
    {"product": "Phone", "price": 800.0, "score": 4.5},  
]
```

```
    {"product": "Charger", "price": 15.0, "score": 4.2}  
]
```

(Note: "Laptop" is excluded since its price of 1200 exceeds 1000.0).

Difficult Assignments

Assignment 5: Stateful Ledger Scope Machine (LEGB Scopes & Closures)

Scenario

You are developing a stateful balance ledger tracker that tracks account state history. To satisfy strict architecture requirements, you must manage this state without defining any classes (`class` keyword is prohibited). Instead, you must use closures, nested functions, and Python scoping variables.

Problem Description

1. Define a global variable `AUDIT_TRANSACTION_COUNT = 0` at the top level of your script.
2. Implement a function `create_bank_account(owner_name, initial_balance)` that returns a dictionary of actions.
 - Inside `create_bank_account`, initialize local variables `balance` (float, set to `initial_balance`) and `history` (list of strings, initially containing `["Account created with 1000.0"]` or similar initial message).
 - Define three nested functions inside `create_bank_account`:
 - **`deposit(amount)`:**
 - Adds `amount` to the local `balance` variable.
 - Appends the string `"deposit <amount>"` to the local `history` list.
 - Increments the global `AUDIT_TRANSACTION_COUNT` by 1 using the `global` keyword.
 - **`withdraw(amount)`:**
 - Checks if the current local `balance` is sufficient (`balance ≥ amount`).
 - If yes, deducts `amount` from `balance`, appends the string `"withdraw <amount>"` to `history`, and increments global `AUDIT_TRANSACTION_COUNT` by 1.
 - If the balance is insufficient, raises a standard `ValueError` with message `"Insufficient balance"`.
 - **`get_statement()`:**
 - Returns a tuple containing `(owner_name, current_balance, history_list_copy)`. (Note: Make sure `history_list_copy` is a copy of the history list to prevent direct external modification).
 - Return a dictionary containing key-value mappings to these inner functions:

```
return {  
    "deposit": deposit,  
    "withdraw": withdraw,
```

```
        "statement": get_statement
    }
```

3. **Constraint:** You must utilize the `nonlocal` keyword to modify the variables `balance` and `history` inside the nested functions.

Example Walkthrough

```
# Initial State
print(AUDIT_TRANSACTION_COUNT) # Output: 0

# Create account
acc = create_bank_account("Arham", 1000.0)

# Deposit
acc["deposit"](200.0)

# Withdraw
acc["withdraw"](150.0)

# Overdraft attempt (should raise ValueError)
try:
    acc["withdraw"](2000.0)
except ValueError as e:
    print(e) # Output: Insufficient balance

# Get statement
owner, bal, txn_history = acc["statement]()
print(owner)      # Output: Arham
print(bal)        # Output: 1050.0
print(txn_history) # Output: ['Account created with 1000.0', 'deposit
200.0', 'withdraw 150.0']

# Verify global log count
print(AUDIT_TRANSACTION_COUNT) # Output: 2
```

Assignment 6: Server Log Analyzer & Traffic Classifier (Advanced RegEx)

Scenario

An automated server monitor analyzes web traffic logs to detect security issues. The monitor extracts HTTP details from log strings and filters out requests originating from local network IP addresses.

Problem Description

Write a function `analyze_server_logs(logs_text)` that parses web logs:

- `logs_text` is a multi-line string containing server logs. Each log line matches this exact format: "`<IP> - - [<timestamp>] \"<HTTP_METHOD> <URL> <HTTP_VERSION>\" <STATUS_CODE> <BYTES>`" Example: "`192.168.1.5 - - [28/Aug/2026:10:00:00] \"GET /index.html HTTP/1.1\" 200 1024`"
- The function must perform the following:
 1. Compile a single regular expression using **named capture groups** to extract:
 - `ip`: The source IP address.
 - `time`: The timestamp value inside the brackets `[]`.
 - `method`: The HTTP method (GET, POST, PUT, DELETE).
 - `resource`: The URL route value (e.g., `/index.html`).
 - `status`: The integer status code.
 - `bytes`: The integer bytes sent.
 2. Parse the input `logs_text` line-by-line using your regex. If a line does not match the format, print a warning: "`Warning: Could not parse line: '<line>'. Skipping.`" and continue to the next line.
 3. **Local IP Address Filtering:**
 - Check the extracted IP address. If the IP starts with "`192.168.`" or "`10.`", it is classified as a local network request.
 - Filter out and ignore local network requests; do not include them in the final list.
 4. For external request logs, compile a dictionary containing: `{"ip": ip, "time": time, "method": method, "resource": resource, "status": status_code, "bytes": bytes_sent}` (Note: `status` and `bytes` must be stored as integers).
 5. Return a list of these dictionaries.

Sample Input

```
log_data = """192.168.1.5 - - [28/Aug/2026:10:00:00] "GET /index.html
HTTP/1.1" 200 1024
8.8.8.8 - - [28/Aug/2026:10:10:00] "GET /api/v1/users HTTP/1.1" 200 4096
Corrupted log entry here
10.0.0.12 - - [28/Aug/2026:10:15:00] "POST /submit_data HTTP/1.1" 403 512
172.16.0.4 - - [28/Aug/2026:10:20:00] "POST /login HTTP/1.1" 401 256"""
```

Expected Output

Console Warnings Printed:

```
Warning: Could not parse line: 'Corrupted log entry here'. Skipping.
```

Returned List:

```
[
  {
    "ip": "8.8.8.8",
```

```
    "time": "28/Aug/2026:10:10:00",  
    "method": "GET",  
    "resource": "/api/v1/users",  
    "status": 200,  
    "bytes": 4096  
  },  
  {  
    "ip": "172.16.0.4",  
    "time": "28/Aug/2026:10:20:00",  
    "method": "POST",  
    "resource": "/login",  
    "status": 401,  
    "bytes": 256  
  }  
]
```

(Note: IP *192.168.1.5* and *10.0.0.12* are skipped because they are local).